Advanced python features - part II

Computing Methods for Experimental Physics and Data Analysis

A. Manfreda

alberto.manfreda@pi.infn.it

INFN–Pisa

Compiled on October 17, 2025

Functions inside functions

- Functions in python are first class object
- The name is a bit misleading, but what it actually means is that functions can be passed as argument to other functions and returned as result from other functions
- This shouldn't surprise you much: functions are objects of a '*function*' class, so they behave like any other vairable in Python
- Another thing you can (and sometimes want to) do is *defining* a function inside another.
- Let's see how it works

Functions inside functions

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/inner_outer.py

```
1  def outer():
2      def inner(): # Defining the inner function inside the outer function
3          print('Inner function')
4          return # End of the inner function
5      return inner # Inner is the output of outer
6
7  my_func = outer() # my_func is now referncing 'inner'
8  print(my_func.__name__)
9  my_func() # Calling my_func is equal to calling 'inner'
10
11 def outer2():
12     some_string = 'Hello!'
13     def inner():
14         # We have access to the variables in the outer function!
15         print(some_string)
16     return inner
17
18 my_other_func = outer2()
19 my_other_func()
20
21 [Output]
22 inner
23 Inner function
24 Hello!
```

Closures and free variables

- When a function is created inside another function it has access to the local variables of the outer function, even after its scope ended
- This is technically possible because those variables are kept in a special space of memory, the closure of the inner function
- Such variables are called free variables
- In this way, a function can maintain a state within its closure
- This makes possible the use of functions for tasks that in other languages are reserved to classes
- Let's take a look at the following example

An inefficient rotation

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/rotation_naive.py

```
1  import numpy
2
3  def rotate(x, y, theta):
4      """ Naive implementation. This works, but is inefficient - we have to
5      calculate cos(theta) and sin(theta) for each pair (x,y)! """
6      c = numpy.cos(theta)
7      s = numpy.sin(theta)
8      x_rot = c * x - s * y
9      y_rot = s * x + c * y
10     return x_rot, y_rot
11
12 x, y = 1., 0.
13 theta = numpy.pi/4
14 print(rotate(x, y, theta)) # Rotation of pi/2
15
16 def efficient_rotate(x, y, c_theta, s_theta):
17     """ Efficient rotation. The user gives cos and sin, so they are not
18     calculated for each pixel """
19     x_rot = c * x - s * y
20     y_rot = s * x + c * y
21     return x_rot, y_rot
22
23 c = numpy.cos(theta)
24 s = numpy.sin(theta)
25 print(efficient_rotate(x, y, c, s)) # The syntax is very ugly!
26
27 [Output]
28 (np.float64(0.7071067811865476), np.float64(0.7071067811865475))
29 (np.float64(0.7071067811865476), np.float64(0.7071067811865475))
```

An efficient rotation

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/rotator.py

```
1  import numpy
2
3  def create_rotator(theta):
4      """ Efficient rotation. Cosinus and sinus values are saved in the closure,
5      so that they are computed exactly once."""
6      c = numpy.cos(theta)
7      s = numpy.sin(theta)
8      def rotate(x, y):
9          x_rot = c * x - s * y
10         y_rot = s * x + c * y
11         return x_rot, y_rot
12     return rotate
13
14 x, y = 1., 0.
15 theta = numpy.pi/4
16 rotate_by_theta = create_rotator(theta)
17 print(rotate_by_theta(x, y))
18
19 [Output]
20 (np.float64(0.7071067811865476), np.float64(0.7071067811865475))
```

A caveat about free variables

- Note: if you *assign* to a free variable in the inner function, by default a new, local variable is created instead!
- To avoid this you have to explicitly declare that you want to access the variable in the closure using the nonlocal keyword
- Remember: *'Explicit is better then implicit'*

Free variables - a mistake to avoid

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/closure_wrong.py

```
1  def running_average():
2      total_count = 0
3      num_elements = 0
4      def accumulator(value):
5          total_count += value # Doesn't work! total_count is reassigned!
6          num_elements += 1 # Doesn't work! total_count is reassigned!
7          return total_count/num_elements
8      return accumulator
9
10 run_avg = running_average()
11 print(run_avg(1.))
12 print(run_avg(5.))
13 print(run_avg(2.5))
14
15 [Output]
16 Traceback (most recent call last):
17   File "/home/alberto/teaching/computing/cmepda/slides/latex/snippets/closure_wrong.py", lin
18     print(run_avg(1.))
19           ^^^^^^^^^^^
20   File "/home/alberto/teaching/computing/cmepda/slides/latex/snippets/closure_wrong.py", lin
21     total_count += value # Doesn't work! total_count is reassigned!
22     ^^^^^^^^^^^
23 UnboundLocalError: cannot access local variable 'total_count' where it is not associated wit
```

Free variables - the correct way

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/closure_right.py

```
1  def running_average():
2      total_count = 0
3      num_elements = 0
4      def accumulator(value):
5          # We declare the relevant variables as nonlocal
6          nonlocal total_count, num_elements
7          # Now we can assign to them - the variables in the closure will be
8          # modified, as we want!
9          total_count += value
10         num_elements += 1
11         return total_count/num_elements
12     return accumulator
13
14 run_avg = running_average()
15 print(run_avg(1.))
16 print(run_avg(5.))
17 print(run_avg(2.5))
18
19 [Output]
20 1.0
21 3.0
22 2.8333333333333335
```

Wrapping functions

- The typical use of defining a function inside a function is to create a wrapper
- A wrapper is a function that calls another one adding a layer of functionalities in between - for example it may do some pre-process of the input, or change the output in some way, or measure the execution time or whatever we want
- The technique for creating a wrapper function in Python is:
  - Pass the function that we want to wrap as argument of the outer function
  - Inside the outer function we define an inner function, which is the actual wrapper
  - The wrapper calls the wrapped function and adds its functionalities, before and/or after the call. It may return the same output or a manipulated one.
  - Then from the outer function we return the wrapper

Wrapper

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/wrapper.py

```
1  def some_function(a, b):
2      print('Executing {} x {}'.format(a, b))
3      return a * b
4
5  def add_n_wrapper(func, n): # We take the wrapped function as argument
6      """ This wrapper adds n to the result of the wrapped function"""
7
8      def wrapper(*args, **kwargs):
9          """We passs the arguments as *arg, **kwargs, because this is the most
10         general form in Python: we can collect any combination of arguments like
11         that. Note that we have access to both 'func' and 'n', as they are stored
12         in the closure of 'wrapper'"""
13         result = func(*args, **kwargs) # Pass the arguments to the wrapped fucntion
14         print('Adding {}'.format(n))
15         return result + n # Return a modified result in this case
16
17     return wrapper # From add_n_wrapper we return the wrapper
18
19 function_plus_five = add_n_wrapper(some_function, 5)
20 print('Result = {}'.format(function_plus_five(2, 3)))
21
22 [Output]
23 Executing 2 x 3
24 Adding 5
25 Result = 11
```

Decorators

- Often, when you wrap a function, you don't want to change it's name, so you reassign the wrapped function to its old name
- In fact, this technique is so common that python introduced a special syntax for it: decorators
- A decorated function has simply the name of the wrapper added with a '@' on top of its declaration

Decorators

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/decorator.py

```
1  def print_function_info(func):
2      def wrapper(*args, **kwargs):
3          print('Calling function \'{}\''.format(func.__name__))
4          print('Positional arguments = {}'.format(args))
5          print('Keyword arguments = {}'.format(kwargs))
6          return func(*args, **kwargs)
7      return wrapper
8
9  @print_function_info
10 def some_function(a, b, c=0):
11     return a * b + c
12
13 # This is equivalent to: some_function = print_function_info(some_function)
14
15 print(some_function(1, 2, c=7))
16  # Inspecting the function reveals that we are calling the wrapper
17 print('The name of the function is \'{}\''.format(some_function.__name__))
18
19 [Output]
20 Calling function 'some_function'
21 Positional arguments = (1, 2)
22 Keyword arguments = 'c': 7
23 9
24 The name of the function is 'wrapper'
```

A decorator to measure execution time

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/time_measuring_decor.py

```
1  import time
2  from functools import wraps
3
4  def clocked(func):
5      """ We use functools.wraps to keep the original function name and docstring"""
6      @wraps(func)
7      def wrapper(*args, **kwargs):
8          tstart = time.perf_counter()
9          result = func(*args, **kwargs)
10         exec_time = time.perf_counter() - tstart
11         print('Function {} executed in {} s'.format(func.__name__, exec_time))
12         return result
13     return wrapper
14
15 @clocked
16 def square_list(input_list):
17     """ Return the square of a list"""
18     return [item**2 for item in input_list]
19
20 # Make sure the function name and docstring look the same
21 print('\'{}\': {}'.format(square_list.__name__, square_list.__doc__))
22 square_list(range(2000000))
23
24 [Output]
25 'square_list':  Return the square of a list
26 Function square_list executed in 0.6440078230007202 s
```

The @classmethod decorator

- We have already seen a built-in Python decorator: *@property*
- We used that to get proper encapsulation
- There is another built-in decorator one which is very useful for classes: *@classmethod*
- A classmethod is like a class attribute: you don't need an instance to use it
- A class method can access class attributes but not instance attributes
- The main use for class methods is to provide alternate constructors

Class method

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/classmethod.py

```
1  import numpy
2
3  class LabData:
4
5    def __init__(self, times, values):
6       """ Our usual constructor"""
7       self.times = numpy.array(times, dtype=numpy.float64)
8       self.values = numpy.array(values, dtype=numpy.float64)
9
10   @classmethod # The classmethod decorator
11   def from_file(cls, file_path): # We get the class as first argument, not self
12       """ Constructor from a file"""
13       print(cls)
14       times, values = numpy.loadtxt(file_path, unpack=True)
15       # We call the constructor of 'cls' which is our LabData
16       # This is not a 'real' constructor, we need to return the object!
17       return cls(times, values)
18
19 # We call the alternate constructor from the class itself, not from an instance!
20 lab_data = LabData.from_file('snippets/data/measurements.txt')
21 print(lab_data.values)
22
23 [Output]
24 <class '__main__.LabData'>
25 [15.2 12.4 11.7 13.2]
```

The @staticmethod decorator

- Another built-in decorator (though less used than *@classmethod*) is *@staticmethod*
- A static method is a method that does not receive the class or instance as first argument
- Because of that, a static method does not alter the state of the class
- In some sense a static method is only loosely coupled to the class - it is defined inside the class body just for convenience (maybe for semantic proximity) but it could be defined outside the class as well

Static method

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/staticmethod.py

```
1  import math
2
3  class Angle:
4
5      # A bunch of useful methods....
6
7      @staticmethod
8      def rad2deg(rad):
9          # The self argument is not passed to a static method
10         return rad * 180./math.pi
11
12     @staticmethod
13     def deg2rad(deg):
14         # The self argument is not passed to a static method
15         return deg * math.pi / 180.
16
17 print(Angle.rad2deg(math.pi/2))
18 print(Angle.deg2rad(45.))
19
20 [Output]
21 90.0
22 0.7853981633974483
```

The @dataclass decorator

- A very useful decorator is *@dataclass*
- A *dataclass* is a class that is designed to only (or mostly) hold data values.
- A dataclass is defined by declaring the name and type of its attributes. You do not have to type any other code, the interpreter will automatically generate the __init__, __eq__ and __repr__ special methods for you.
- You can customize what methods are added to your dataclass through a number of flags: *init*, *repr*, *eq*, *order*, *unsafe_hash*, *frozen* ... (see the full list at `https://docs.python.org/3/library/dataclasses.html#dataclasses.dataclass`)

Dataclass

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/dataclass.py

```python
from dataclasses import dataclass

# Traditional way
class PersonOldStyle:
    def __init__(self, name, age, height):
        self.name = name
        self.age = age
        self.height = height
    def __repr__(self):
        return f'{self.__class__.__name__}(name=\'{self.name}\', age={self.age}, '\
               f'height={self.height})'
    def __eq__(self, other):
        return (self.name, self.age, self.height) == \
               (other.name, other.age, other.height)

# With dataclass
@dataclass
class PersonDataclass:
    name : str # Notice the type hint (which is mandatory but not enforced)
    age : int
    height : float

Joe = PersonOldStyle('Joe', 35, 1.84)
print(Joe)
Mary = PersonDataclass('Mary', 28, 1.62)
print(Mary)

[Output]
PersonOldStyle(name='Joe', age=35, height=1.84)
PersonDataclass(name='Mary', age=28, height=1.62)
```

Passing arguments to a decorator

- Making a decorator that accepts arguments is somewhat more complex
- The basic idea is that you need to add yet another level: a *decorator factory* function
- So you now end up with three levels:
  - The decorator factory that takes the parameters for the decorators, creates it and retruns it
  - The decorator that takes as input a function and returns the wrapper
  - The wrapper that implements the actual additional functionalities; usually takes as input the same parameters as the decorated/wrapped function and returns its results (if any)

Decorception

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/decorator_factory.py

```python
from functools import wraps

# This is how a generic decorator with arguments looks like

def decorator_factory(*params): # The signature can be anything
    def decorator(function):
        @wraps(function)
        def wrapper(*args, **kwargs):
            print(f'Decorator arguments: {params}')
            # some preprocess here
            result = function(*args, **kwargs)
            # some post-process here
            return result
        return wrapper
    return decorator

# usage
@decorator_factory(1, 2, 3)
def f(x):
    return x**2

f(5)

[Output]
Decorator arguments: (1, 2, 3)
```

Repeat

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/decorator_repeat.py

```
1  from functools import wraps
2
3  def repeat(num_times):
4      """ Repeat a function call a given number of times"""
5      def decorator(function):
6          @wraps(function)
7          def wrapper(*args, **kwargs):
8              for _ in range(num_times):
9                  function(*args, **kwargs)
10         return wrapper
11     return decorator
12
13 # usage
14 @repeat(num_times=4)
15 def greet():
16     print('Hello!')
17
18 greet()
19
20 [Output]
21 Hello!
22 Hello!
23 Hello!
24 Hello!
```

Customizing dataclass methods

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/dataclass_customized.py

```
1  from dataclasses import dataclass
2
3  @dataclass(order=True, frozen=True)
4  class Person:
5      name: str
6      age: int
7      height: float
8
9  Joe = Person('Joe', 32, 1.84)
10 Mary = Person('Mary', 25, 1.62)
11 print(Joe > Mary)
12 Joe.age = 33
13
14 [Output]
15 False
16 Traceback (most recent call last):
17   File "/home/alberto/teaching/computing/cmepda/slides/latex/snippets/dataclass_customized.p
18     Joe.age = 33
19     ^^^^^^^
20   File "<string>", line 4, in __setattr__
21 dataclasses.FrozenInstanceError: cannot assign to field 'age'
```

Chaining decorators

https://github.com/lucabaldini/cmepda/tree/master/slides/latex/snippets/decorator_chain.py

```
1  from decorator_samples import clocked, repeat, print_function_info
2
3  """ To chain the effect of more than one decorator just stack them above the
4  function definition """
5
6  @print_function_info
7  @repeat(num_times=3)
8  @clocked
9  def greet(name):
10     print(f'Hello {name}')
11
12 greet('Bob')
13
14 [Output]
15 Calling function 'greet'
16 Positional arguments = ('Bob',)
17 Keyword arguments =
18 Hello Bob
19 Function greet executed in 5.208999937167391e-06 s
20 Hello Bob
21 Function greet executed in 4.772000465891324e-06 s
22 Hello Bob
23 Function greet executed in 2.697001036722213e-06 s
```